



## TECHNICAL SPECIFICATION

# Stewardex Mobile Application

## ■ Frontend Developer Guide

---

|         |              |
|---------|--------------|
| Release | Phase 1      |
| Date    | 15 June 2026 |
| Status  | For review   |

## ■ 1. Recommended Stack

| Concern        | Recommendation                                                     | Notes                                                                                                                                                                                                                   |
|----------------|--------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Framework      | <b>React Native (Expo, managed workflow)</b>                       | Matches the web team's React skills; Expo simplifies push, OTA updates, and builds. Bare RN is acceptable if native modules are needed.                                                                                 |
| Language       | TypeScript                                                         | Strongly recommended for API typing.                                                                                                                                                                                    |
| Navigation     | React Navigation (native stack + bottom tabs)                      | Tab bar for the main sections, stacks per feature.                                                                                                                                                                      |
| Server state   | <b>TanStack Query (React Query)</b>                                | Same model as web ( <code>@tanstack/react-query</code> ); reuse cache/stale conventions.                                                                                                                                |
| Client state   | <b>Zustand</b>                                                     | Mirror the web <code>authStore</code> pattern ( <code>user</code> , <code>token</code> , <code>orgId</code> , <code>isAuthenticated</code> ). Persist with <code>expo-secure-store</code> / <code>AsyncStorage</code> . |
| HTTP           | <b>Axios</b>                                                       | Single instance + interceptors (see §4).                                                                                                                                                                                |
| Real-time      | <b>socket.io-client</b> 4.x                                        | For chat, presence, notifications.                                                                                                                                                                                      |
| Push           | <b>expo-notifications</b> (Expo push) or FCM + APNs directly       | See §7. Requires new backend route for device-token registration.                                                                                                                                                       |
| Forms          | React Hook Form + Zod                                              | Same as web.                                                                                                                                                                                                            |
| Secure storage | <code>expo-secure-store</code>                                     | Store JWT + orgId securely, not in plain AsyncStorage.                                                                                                                                                                  |
| File upload    | <code>expo-image-picker</code> / <code>expo-document-picker</code> | For expense invoices and profile pictures (multipart).                                                                                                                                                                  |
| PDF / docs     | <code>react-native-pdf</code> or in-app browser                    | For policy documents and exported registers.                                                                                                                                                                            |
| Dates          | <code>date-fns</code> / <code>dayjs</code>                         | Same as web.                                                                                                                                                                                                            |

The web app is **React 19 + Vite + React Router + TanStack Query + Zustand + Axios + socket.io-client + Tailwind**. Reusing the same data-layer libraries keeps API logic portable — much of the web's `features/<area>/*.api.js` modules can be ported almost verbatim.

## ■ 2. App Architecture

```
src/
├─ api/                # Axios instance + per-feature api modules (port from web)
│  ├─ client.ts        # configured axios + interceptors
│  ├─ auth.api.ts
│  ├─ calendar.api.ts
│  ├─ meetings.api.ts
│  └─ approvals.api.ts # ... one per feature
├─ store/
│  ├─ authStore.ts     # user, token, orgId, isAuthenticated (persisted, secure)
│  └─ notificationStore.ts
├─ navigation/
│  ├─ RootNavigator.tsx # auth stack vs. app stack switch
│  ├─ TabNavigator.tsx  # bottom tabs
│  └─ linking.ts         # deep links from push notifications
├─ features/           # one folder per feature (screens + hooks)
│  ├─ calendar/ meetings/ tasks/ chat/ approvals/
│  ├─ policies/ risk/ coi/ complaints/ expenses/
│  └─ billing/ support/ settings/ notifications/
├─ components/         # shared UI primitives (Button, Card, Chip, Avatar, ...)
├─ lib/                # socket, push registration, permissions helpers
└─ config.ts           # API base URL, env
```

Group code **by feature**, mirroring the web's `src/features/<area>/` layout, so logic maps 1:1 between the two clients.

## ■ 3. Environment & Config

```
// config.ts
export const API_BASE_URL =
  process.env.EXPO_PUBLIC_API_BASE_URL ?? 'http://localhost:5000/api/v1';
```

- Backend base path is `/api/v1`. Platform (tenant) endpoints live under `/api/v1/platform/...`; auth lives under `/api/v1/auth/...`.
- The [Socket.IO](#) server runs on the **same origin** as the API (path `/socket.io`).

## ■ 4. API Client & Interceptors

Port the web's single-axios-instance pattern. **Every authenticated request must carry two headers:**

```
Authorization: Bearer <jwt>
x-org-id: <orgId>
```

```
// api/client.ts
import axios from 'axios';
import { API_BASE_URL } from '../config';
import { useAuthStore } from '../store/authStore';

export const api = axios.create({ baseUrl: API_BASE_URL });

api.interceptors.request.use((cfg) => {
  const { token, orgId } = useAuthStore.getState();
  if (token) cfg.headers.Authorization = `Bearer ${token}`;
  if (orgId) cfg.headers['x-org-id'] = orgId;
  // For FormData, let axios/RN set the multipart boundary – don't force JSON content-type.
  return cfg;
});

api.interceptors.response.use(
  (res) => res.data, // backend wraps as { success, data } – unwrap to body
  (err) => {
    const { status, data } = err.response ?? {};
    const code = data?.code;
    if (status === 401 && !isMfaCode(code)) {
      useAuthStore.getState().logout(); // navigate to Login
    }
    // Re-throw normalized error: { message, code, status, details? }
    return Promise.reject({
      message: data?.message ?? err.message,
      code, status, details: data?.details,
    });
  },
);
```

### Key conventions (identical to web):

- Responses are wrapped `{ success, data }` (or `{ error }`); unwrap to `data` in the response interceptor. API functions return the body directly.
- Errors thrown to feature code have shape `{ message, code, status, details? }` — branch on `err.code`, not the raw HTTP status.
- On **401** → log out and route to Login, **unless** the code is an MFA challenge ( `MFA_REQUIRED` | `MFA_INVALID` | `MFA_EXPIRED` ), which the login flow must catch and handle.
- On **402** (limit reached) and **403** (feature locked / `WORKFLOW_NOT_CONFIGURED` ) → show an appropriate “upgrade / not available on mobile” message. Workflow-config and limit-config actions are web-only; surface a friendly message rather than a dead end.
- **Auditors are read-only.** If `user.is_auditor === true`, disable all create/approve/submit actions in the UI.

## ■ 5. Authentication Flow

The backend uses **JWT + OTP**, with optional **TOTP MFA**.

1. POST /auth/login { email, password }  
→ returns either a session needing OTP, or (if MFA challenge) an MFA code error.  
Response includes user, user\_id, orgId.
2. POST /auth/otp/verify { user\_id, code, orgId }  
→ returns { token, user, orgId }
3. setAuth(user, token, orgId) → persist to secure store → enter app.

- Store login\_pending\_email / a transient session between step 1 and 2.
- The JWT payload carries { userId, orgId, roles, permissions, isAuditor }.
- **Permissions** arrive as an array of strings on user.permissions, in three forms:
  - module-scoped module:<id>:view|edit|delete
  - action-scoped <resource>:<action> (e.g. policy:acknowledge, expense\_workflow:create)
  - wildcard \*: (admin/owner — bypasses all checks)
- Build a useModulePermissions(moduleId) helper returning { canView, canEdit, canCreate, canDelete }, mirroring the web. Auditors always resolve to view-only.
- Fetch live permissions with GET /auth/me/permissions after login if needed.

**Auth screens:** Login, OTP entry (4-digit auto-submit), Forgot Password ( POST /auth/forgot-password → POST /auth/reset-password ), and the MFA challenge step during login.

## ■ 6. Real-Time (Socket.IO)

Connect once after authentication. Used for **chat, presence, and live updates**.

```
import { io } from 'socket.io-client';
const socket = io(SOCKET_ORIGIN, { auth: { token } }); // JWT verified on handshake
```

**Rooms** (server-managed): org:<orgId>, user:<orgId>:<userId>, chan:<channelId>.

**Client → Server events:** chat:join (channelId), chat:leave (channelId), chat:typing ({ channelId }).

**Server → Client events to handle:**

| Event                               | Payload                                      | Use                                  |
|-------------------------------------|----------------------------------------------|--------------------------------------|
| <code>chat:message:new</code>       | message object                               | Append to channel; bump unread/badge |
| <code>chat:message:edited</code>    | updated message                              | Replace in list                      |
| <code>chat:message:reaction</code>  | <code>{ messageId, emoji, added }</code>     | Update reactions                     |
| <code>chat:read</code>              | <code>{ messageIds, by_user_id }</code>      | Update read receipts                 |
| <code>chat:typing:user</code>       | <code>{ channelId, userId, user, at }</code> | Typing indicator                     |
| <code>chat:presence</code>          | <code>{ userId, online }</code>              | Online/offline dots                  |
| <code>chat:presence:snapshot</code> | <code>{ userIds: [...] }</code>              | Initial presence on connect          |

Join `chat:join` on opening a channel; leave on close. Reconnect on app foreground and refetch recent messages to catch up after backgrounding (the web uses refetch-on-focus for the same reason).

## ■ 7. Push Notifications

The current backend implements **Web Push (VAPID)** for browsers only — this does **not** work for native iOS/Android. The mobile app needs native push via **FCM (Android) + APNs (iOS)**, easiest through **Expo push tokens**.

### Mobile responsibilities:

1. Request notification permission on first relevant launch.
2. Obtain the device push token (Expo token or raw FCM/APNs).
3. Register it with the backend (**new endpoint** — see Backend Guide): `POST /platform/notifications/push/register-device` with `{ token, platform: 'ios' | 'android', deviceId }`.
4. Unregister on logout: `POST /platform/notifications/push/unregister-device`.
5. Handle taps via deep links → navigate to the related approval / meeting / chat / policy. Each notification carries `type`, `entity_type`, `entity_id` (see notification types below).
6. Keep the in-app **Notification Center** in sync via `GET /platform/notifications` and `PATCH /platform/notifications/:id/read`.

Coordinate with the backend developer: device-token registration and an APNs/FCM (or Expo) send path are the **main backend additions** required for the mobile app.

**Notification types** (from web) include: `approval_pending`, `workflow_approved`, `workflow_rejected`, `returned_for_resubmission`, `meeting_invitation`, `meeting_notes_added`, `chat_mention`, `ticket_assigned`. Map each to an icon/tone and a deep-link target.

## ■ 8. Screen-by-Screen Build Guide

Each feature below lists its **primary endpoints** and notes. Base path `/api/v1/platform` is omitted for brevity except where ambiguous. Full request/response shapes are visible in the web `*.api.js` modules and the Postman collection in the backend repo ( `/postman` ).

### 8.1 Calendar

- `GET /calendar` — all events (custom + system deadlines: policy reviews, training, registrations).
- `POST /calendar` , `PUT /calendar/:id` , `DELETE /calendar/:id` .
- Views: month / week / day / list (default to **list** on phones). Color-code by event `type` . “Expiring soon” = filter by upcoming due dates. Mark calendar tasks complete via `PUT` .

### 8.2 Meetings

- `GET /meetings` , `GET /meetings/:meetingId` .
- `POST /meetings` (create), `PUT /meetings/:meetingId` , `POST /meetings/:meetingId/cancel` .
- `POST /meetings/:meetingId/notes` (minutes), `POST /meetings/:meetingId/attendance` (RSVP: invited/confirmed/declined/attended).
- Group by Today / This Week / Later / Past. Show type chip (board/general/resolution), status, location, attendee avatars, important flag.

### 8.3 My Tasks

- **Aggregated client-side** (as on web) from three sources:
  - pending approvals — `GET /approvals`
  - today's meetings — `GET /meetings`
  - calendar tasks — `GET /calendar`
- Priority flags are stored locally (web uses `localStorage` → use `AsyncStorage`). Completion: approvals/meetings via their actions, calendar tasks via `PUT /calendar/:id` .
- (Optional: the backend team may add a single `GET /platform/me/tasks` aggregation endpoint — see *Backend Guide* — to simplify this. Build client-side aggregation as the default.)

### 8.4 Chat

- `GET /chat/channels` — channel list.
- `GET /chat/channels/:channelId/messages` — paginated history.
- `POST /chat/channels/:channelId/messages` — send (body ± attachments via multipart).
- `PATCH /chat/messages/:messageId` (edit), `DELETE /chat/messages/:messageId` (soft delete).
- `POST /chat/channels/:channelId/read` , `GET /chat/messages/:messageId/reads` .
- `POST /chat/messages/:messageId/reactions` (toggle emoji), `.../pin` , `.../star` ; `GET /chat/channels/:channelId/pinned` .
- Combine REST (history, send) with [Socket.IO](#) (live new/edit/reaction/read/typing/presence). Three-pane web layout collapses to **list** → **conversation** navigation on mobile.

## 8.5 Approval Workflow

- `GET /approvals` — requests (to approve / awaiting / past).
- `GET /approvals/:approvalId` — detail with step chain.
- `POST /approvals/:approvalId/approve` ({ comment? }), `POST /approvals/:approvalId/reject` ({ reason }).
- `GET /approvals/precheck` — check a workflow exists before offering a submit action.
- Show approval type (sequential/parallel), step progress, current approver, amount for financial requests.

## 8.6 Policies (View & Acknowledge)

- `GET /policies` (filter: status, category, search), `GET /policies/counts`.
- `GET /policies/:policyId` — detail + version history; render the document (PDF viewer / in-app browser via signed URL).
- `POST /policies/:policyId/acknowledge` — record sign-off.
- `GET /policies/:policyId/acknowledgements` — who acknowledged + when.

## 8.7 Risk Register (View)

- `GET /risks` (filter: status, category, owner), `GET /risks/counts`, `GET /risks/:riskId`.
- View-only on mobile: list + detail, residual/inherent scores, likelihood × consequence. (No create/edit in Phase 1.)

## 8.8 COI Register (View + Declare)

- `GET /coi/list`, `GET /coi/pending`, `GET /coi/:coiRequestId`.
- `POST /coi/declare` — user submits their own declaration (enters approval workflow).
- Approve/reject endpoints exist but are typically web-side; include if role permits.

## 8.9 Complaint Register (View)

- `GET /complaints`, `GET /complaints/stats/overview`, `GET /complaints/:complaintId`.
- View-only on mobile: list + detail, status, assignee. (No create/manage in Phase 1.)

## 8.10 Expenses

- `POST /expenses` — submit (triggers approval workflow if configured).
- `POST /expenses/upload` — upload invoice to S3 (multipart); `GET /expenses/file-url` — signed download URL.
- `GET /expenses`, `GET /expenses/:expenseId`, `PUT /expenses/:expenseId`, `DELETE /expenses/:expenseId`.
- Use `expo-image-picker` / `document-picker` for the invoice. Supplier and project selection pull from their respective lists (carry over web field set: amount, category, supplier, project/funding agreement, asset flag).

## 8.11 Billing

- `GET /billing/me` — current plan, limits, feature flags.



- `GET /billing/usage` — usage vs. limits (soft/hard caps, overage). ( `/usage/stream` SSE is optional on mobile; polling is fine.)
- `GET /billing/available-plans` , `GET /billing/active-discount` , `GET /billing/events` .
- `POST /billing/checkout` → open the returned Stripe URL in an in-app browser; `POST /billing/portal` → open Stripe customer portal. Handle return deep link and refetch `/billing/me` .
- Plan changes ( `change-plan` , `switch-cycle` , `cancel` , `restart` , `hard-cap` ) can be exposed if desired; at minimum support view + upgrade via Stripe.

## 8.12 Help Center

- `GET /support-tickets` , `POST /support-tickets` , `GET /support-tickets/:ticketId` , `POST /support-tickets/:ticketId/messages` .
- Ticket list with status filters; detail with conversation thread + reply box.

## 8.13 Settings (Profile, MFA, Password)

- `GET /me/profile` , `PATCH /me/profile` (name/email), `POST /me/profile/picture` (multipart).
- MFA (TOTP): `POST /me/profile/mfa/totp/setup` → render QR code + secret; `POST /me/profile/mfa/totp/enable` , `.../verify` . Disable via `POST /auth/mfa/disable` .
- Password change via the reset flow / a profile password endpoint (confirm exact route with backend).

## 8.14 Notifications

- `GET /platform/notifications` (with unread count), `PATCH /platform/notifications/:id/read` .
- Device push registration: **new** `POST /platform/notifications/push/register-device` (see §7 / Backend Guide).
- Render type-coloured rows, filter by type, deep-link to the related entity on tap.

# ■ 9. Navigation & Information Architecture

Suggested **bottom tab bar** (5 tabs) with the rest behind a “More” menu:

```
[ Home/Tasks ] [ Chat ] [ Approvals ] [ Calendar ] [ More ]
My Tasks      Approval      Meetings      Policies, Risk, COI,
+ Notifications  Workflow      + Calendar  Complaints, Expenses,
bell                                                  Billing, Help, Settings
```

- A persistent **notification bell** (unread count) in the header.
- Deep links from push: `stewardex://approvals/:id` , `stewardex://meetings/:id` , `stewardex://chat/:channelId` , `stewardex://policies/:id` .

## ■ 10. Cross-Cutting Requirements

---

- **Multi-tenant:** never issue an authenticated request without `x-org-id`. If you bypass the axios instance, set both headers manually.
  - **Permissions & auditor mode:** gate every action button by permission; auditors are view-only everywhere.
  - **Feature flags:** plans enable/disable modules. `GET /billing/me` returns `feature_flags`; hide or lock features not enabled for the tenant (use `GET /billing/feature-catalog` for labels).
  - **Error UX:** branch on `err.code`; show friendly messages for `WORKFLOW_NOT_CONFIGURED` (configure on web), limit-reached (402), feature-locked (403).
  - **Empty/loading/offline states** for every list screen.
  - **Secure token storage** ( `expo-secure-store` ), and clear it fully on logout (also unregister push + disconnect socket).
- 

## ■ 11. Reference Material

---

- **Postman collection:** `D:\charitycompliance-backend\postman` — concrete request/response examples for every endpoint.
- **Web API modules:** `D:\charitycompliance-frontend\src\features\<area>*.api.js` and `src/lib/api.js` — portable request logic and exact payload shapes.
- **Web CLAUDE.md:** `D:\charitycompliance-frontend\CLAUDE.md` — auth/permission/interceptor conventions described in detail.
- **Companion:** “Stewardex Mobile — Backend Developer Guide” for the new endpoints required.